Replication and Predictable-Drift Diagnostics for FTS-Diffusion

Anonymous Author(s)
Affiliation
Address
email

Abstract

We revisit FTS-Diffusion (ICLR 2024), a financial time-series generator built from scale-invariant subsequence clustering (SISC), pattern-conditioned diffusion, and learned pattern evolution, and try to reproduce it from the released code. In our runs the released sampling pipeline is near-degenerate: synthetic S&P 500 paths are one repeated motif tiled into an almost perfectly linear price ramp (linear-fit $R^2 \approx 1.0$), independent runs are nearly identical, and synthetic return volatility is about $20\times$ too small. We trace this to three choices in the released sampling code: a deterministic argmax rollout of the pattern chain, a one-unit decoder hidden state, and a commented-out magnitude normalization. We are explicit about confounds we cannot rule out: no checkpoints are shipped, and our training budget is reduced. Consistent with degeneracy, a continuous-rollout Train-on-Augmentation-Test-on-Real (TATR) setup appears to improve S&P 500 forecasting by up to 85%, yet the same protocol never beats its baseline on GOOG or ZC=F. A coincidental scale match would produce exactly this non-transfer, whereas augmentation that carried real structure would help across assets. On the released artifacts the authors' own Kolmogorov–Smirnov and Anderson–Darling return tests cleanly separate synthetic from real in essentially every window, so a simple return check detects the degeneracy that the downstream MAPE misses. We also do not reproduce the Appendix B.3 SISC toy targets. As a predeclared extension we add a predictable-drift diagnostic that calibrates on real assets, tracks exploitable artificial alpha in a 500-seed experiment, and improves a 1000-seed augmentation-selection benchmark. We present these as observations from one reproduction attempt rather than a verdict on the method.

1 Introduction

Deep generative models are attractive for finance because historical market data are scarce, noisy, nonstationary, and not experimentally reproducible. Financial returns also exhibit heavy tails, weak raw-return autocorrelation, volatility clustering, and recurring shapes at irregular time scales [Cont, 2001]. Sequence generators try to preserve temporal structure with adversarial, optimal-transport, and score-based objectives [Yoon et al., 2019, Xu et al., 2020, Tashiro et al., 2021]; financial generators additionally need to respect stylized facts and economically meaningful dependence [Wiese et al., 2020]. FTS-Diffusion [Huang et al., 2024] addresses this by discovering variable-length motifs and generating new trajectories motif by motif.

We position this work as a replication and diagnostic extension rather than a new generator. We retrace the released pipeline and try several generation protocols to see whether any rollout makes the method produce usable samples, check whether the SISC toy-data targets are reproducible, and test whether a simple predictable-drift statistic catches artificial alpha that marginal stylized facts miss. Its architecture is well motivated, but the released sampling pipeline, in our hands, produces

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

degenerate artifacts: the synthetic series collapses to a near-linear price ramp built from one repeated motif (Section 4). Given that collapse, the apparent S&P 500 gain looks less like useful augmentation than a coincidence that does not transfer across assets. We cannot fully separate this from a known confound: the original authors release no trained checkpoints, so we train every module ourselves and at a reduced budget relative to their defaults.

We therefore extend the evaluation with a predictable-drift diagnostic. Under a fixed information vector

$$\omega_t = (1, r_t, r_{t-1}, |r_t|, |r_{t-1}|)^\top, \tag{1}$$

we measure whether the next return r_{t+1} has a nonzero linear conditional mean under a bounded test class. We intend this not as a proof of market efficiency or absence of arbitrage, but as a finite, reproducible check for an important failure mode: a synthetic generator can match tails and volatility clustering while introducing a simple, exploitable conditional drift.

Contributions. We make five contributions. (i) A code-level diagnosis of the released sampling pipeline (Section 4): three specific choices that drive the synthetic series to a near-degenerate linear ramp, plus the underspecifications we had to resolve. (ii) A 30-seed S&P 500 TATR study, with the cross-asset observation that the continuous-rollout gain does not transfer to GOOG or corn futures (ZC=F), which we read as a scale coincidence. (iii) A return-distribution check showing that the authors' own Kolmogorov–Smirnov and Anderson–Darling tests detect the degenerate output that the downstream MAPE still rewards. (iv) A documented Appendix B.3 SISC toy-data gap. (v) A fixed predictable-drift diagnostic, calibrated on real assets, applied to ordered FTS-Diffusion samples, validated by a 500-seed predictive-alpha experiment, and used in a 1000-seed augmentation-selection benchmark. Throughout, we separate claims about the released artifacts from what would need the authors' checkpoints to settle.

2 Background: FTS-Diffusion

FTS-Diffusion decomposes financial time-series generation into three modules [Huang et al., 2024]. Scale-invariant subsequence clustering (SISC) segments a univariate series into variable-length motifs using dynamic time warping (DTW) [Sakoe and Chiba, 1978] and DTW barycenter-style centroid updates [Petitjean et al., 2011]. Each segment is summarized by a pattern identity p_m, a duration scale α_m, and a magnitude scale β_m. A pattern generation module (PGM) then produces concrete segments from these motif conditions; it pairs a scaling autoencoder (Scaling-AE), which encodes and rescales segment shapes, with a pattern-conditioned diffusion model (PCDM) trained on a DDPM-style denoising objective [Ho et al., 2020].

Long-horizon behavior is controlled by the pattern-evolution module (PEM), which learns

$$(p_m, \alpha_m, \beta_m) \mapsto (p_{m+1}, \alpha_{m+1}, \beta_{m+1}), \tag{2}$$

with a classification loss for the next pattern and regression losses for the next duration and magnitude scales. Although useful, this separation makes downstream results sensitive to how the learned state chain is initialized, reset, and divided into synthetic training blocks.

3 Replication protocol

Assets and splits. Our main datasets are daily Yahoo Finance close-price histories for S&P 500, GOOG, and ZC=F. S&P 500 spans 1980–2020 and yields 10,088 trading days and 2,282 post-SISC segments with $K = 14$. GOOG spans 2004–2020 with 3,776 trading days and 733 post-SISC segments with $K = 11$. ZC=F spans 2000–2020 with 4,764 trading days and 629 post-SISC segments with $K = 11$. We follow a chronological train/test design: motif extraction, fitted generator components, standardizers, and downstream predictors are fit only on their allowed training windows, and final scores are computed on held-out real data.

Downstream benchmark. Our main downstream benchmark is TATR: Train on Augmentation, Test on Real. We divide the held-out real slice into an initialization portion and a final real test portion. A one-day-ahead LSTM forecaster is trained on the real initialization portion plus Y synthetic trading years, where one synthetic year has 252 days and Y is swept across a fixed grid. We score with mean

absolute percentage error (MAPE) on the final real test portion. We use paper-like LSTM settings: one-day ahead, window size 64, hidden size 32, two layers, MAE training loss, Adam with learning rate 10^{-2}, and 100 epochs for the primary S&P 500 runs.

Synthetic rollout variants. We compare three ways to produce the synthetic augmentation path, each sharing the same data budget. We use the descriptive name with the code identifier in parentheses.

- **Independent fixed blocks** (`authors`): independent 252-day blocks, each generated from the same fixed initial state.
- **Continuous chunked** (`split`): one continuous trajectory from the pattern-evolution chain, cut into 252-day chunks for downstream training.
- **Independent blocks with random init** (`random_init`): independent 252-day blocks, each from a randomly drawn initial segment.

These three protocols differ only in how the Markov chain transitions across blocks. Because the pattern-evolution module is the only component that carries long-horizon state across motifs, block connectivity is what they probe.

Compute. Our primary 30-seed S&P 500 downstream batch used stored SISC artifacts and PGM/PEM checkpoints and recorded $10{,}478$ seconds on an Apple M4 laptop with 10 CPU cores and 16 GB memory. Lightweight drift diagnostics ran on the same machine: the ordered FTS audit recorded 629.82 seconds with a four-thread cap, the 500-seed predictive-alpha script recorded 2.83 seconds, and the 1000-seed selection benchmark recorded 32.69 seconds. These times exclude earlier exploratory runs not used for the reported tables.

4 Degeneracy of the released generation pipeline

We reproduce FTS-Diffusion end to end from the released code. Before downstream scores, we report what the generator actually produces, because it determines how the later numbers should be read. In our runs the released sampling path is near-degenerate: the synthetic price series is a single repeated motif tiled into an almost perfectly straight line. We give three jointly responsible choices, with file references into our working copy (`fts-diffusion-ref/`), then show the degeneracy on the saved trajectories and the return distribution. We treat these as observations about the released artifacts under our setup rather than proofs about the method; Section 4.5 lists the confounds we cannot rule out.

4.1 Three choices in the released sampling code

(1) Deterministic argmax rollout of the pattern chain. FTS-Diffusion rolls out the pattern-evolution module (PEM) by taking `argmax` for the next pattern and the next length, and the raw regression mean for the next magnitude, with no sampling (`fts-diffusion-ref/models/sampling.py`, lines 30, 33, 34):

```
pred_pattern   = torch.argmax(probabilities_pattern, dim=1).unsqueeze(0)
pred_length    = torch.argmax(probabilities_length,  dim=1).unsqueeze(0) + l_min
pred_magnitude = pred[:, n_patterns+range_length:].float()
```

An autoregressive map $\text{state}_{m+1} = f(\text{state}_m)$ that is deterministic must, iterated from a fixed initial state, converge to a fixed point or a short cycle. Because the paper does not specify the rollout policy (greedy vs. sampling), this greedy reading of the release is defensible, but it removes the stochasticity the Markov-chain description suggests.

(2) One-unit decoder hidden state. Scaling-AE uses an LSTM decoder whose hidden dimension is fed from `pgm_params['sae_hidden_dim']`, set to 1 in the released configuration (`model_params.py`; the decoder LSTM is defined in `scaling_autoencoder.py`). With a single hidden unit the decoder has almost no capacity, and we observe it collapsing to a fixed output that ignores both the conditioning pattern and the diffusion latent.

(3) Commented-out magnitude normalization. A sampling step that would rescale the generated shape to unit range before multiplying by the magnitude β is commented out (`sampling.py`, lines 53–55), so the emitted segment has magnitude $\beta \times$ (whatever range the decoder produced) rather than β. In our runs that range is on the order of 0.07, far below the intended scale, which removes the magnitude information β was meant to carry.

4.2 What the saved trajectories look like

Figure 1 contrasts a real S&P 500 path with one released-pipeline synthetic path. That synthetic series is visually a straight line; a least-squares linear fit gives $R^2 \approx 0.99999$. Three quantitative observations accompany the figure. First, independent runs of a configuration are nearly identical, consistent with a deterministic rollout. Second, the PEM state sequence collapses (Table 1 and Figure 2): on S&P 500 a single pattern is used in 99.6% of segments (a fixed point), while GOOG and ZC=F settle into period-2 cycles. Third, inspecting the trained PGM decoder pattern by pattern, the decoder output is near-constant across all patterns (cross-pattern standard deviation < 0.001): an up-trend pattern and a down-trend pattern yield essentially the same segment. Together these tile one tiny upward-creeping segment across the horizon, producing exactly the linear ramp in Figure 1.

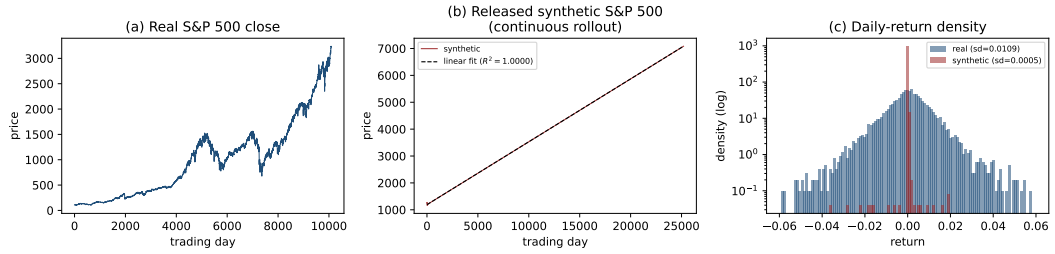

Figure 1: Released-pipeline output on S&P 500. (a) Real close prices show rich structure. (b) One synthetic continuous-rollout path is almost perfectly linear (linear-fit $R^2 \approx 0.99999$). (c) Daily-return density (log scale): synthetic returns concentrate near a single value with standard deviation about $20\times$ smaller than real returns. We show one representative run; runs are nearly identical.

Table 1: PEM state-sequence collapse under the deterministic rollout, from saved `run_*_states.npy`. "Dominant pattern share" is the fraction of generated segments using the most frequent pattern.

Asset (K)	Segments	Dominant pattern share	Regime
S&P 500 (14)	2520	99.6%	fixed point
GOOG (11)	2520	50.0%	period-2 cycle
ZC=F (11)	1867	50.0%	period-2 cycle

4.3 A return-distribution check detects the degeneracy

A natural and direct check is the authors' own return-distribution test. Following their `distribution_tests` routine, we form simple returns and compare real against synthetic with a Kolmogorov–Smirnov (KS) two-sample test and an Anderson–Darling (AD) k-sample test, where a higher p-value means the two return samples are harder to distinguish. Huang et al. report values consistent with indistinguishability (KS around 0.32, AD around 0.12). On the released artifacts the test instead separates the two cleanly (Table 2): across 300 windows per asset and protocol the synthetic and real returns differ in essentially every window, with mean KS p-values at the 10^{-4} to 10^{-7} level and no window passing at $p > 0.05$. Return volatility is the mechanism: the released synthetic series has 4 to $60\times$ too little of it, because a near-linear ramp has almost no daily variation. A degenerate generator is expected to fail a distributional test, so failing one is no surprise; the useful contrast is that this simple return check catches the degeneracy while the downstream price-MAPE of Section 5 still rewards the same output.

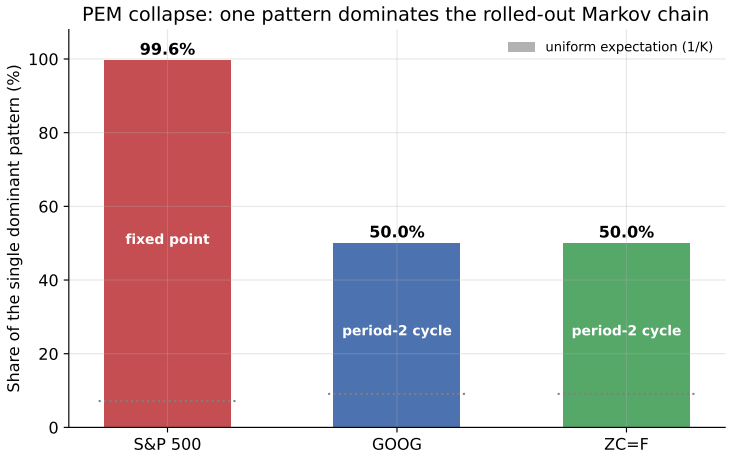

Figure 2: PEM state-sequence collapse under the deterministic rollout. Bars show the share of generated segments that use the single most frequent pattern: 99.6% on S&P 500 (a fixed point) and 50% on GOOG and ZC=F (period-2 cycles). A dashed line marks the uniform expectation $1/K$ that a non-degenerate chain would approach.

Table 2: Return-distribution check on the released artifacts. "Mean KS p" and "% indist." are over 300 windows of length 60; higher is more similar (the paper reports KS around 0.32). "Real/Syn. sd" are daily-return standard deviations. `continuous` denotes an un-chunked continuous rollout, `authors` the independent-block protocol.

Asset	Protocol	Real sd	Syn. sd	Mean KS p	% indist.
S&P 500	authors	0.0109	0.0045	9×10^{-4}	0
S&P 500	continuous	0.0109	0.00046	4×10^{-7}	0
GOOG	authors	0.0184	0.0026	6×10^{-5}	0
GOOG	continuous	0.0184	0.00029	4×10^{-7}	0
ZC=F	authors	0.0181	0.0028	2×10^{-5}	0
ZC=F	continuous	0.0181	0.00041	1×10^{-6}	0

4.4 Underspecifications we had to resolve

Several choices that matter for the result are not pinned down by the paper. Table 3 lists the ones we encountered, separating the three that are load-bearing for the degeneracy above from the compute-driven confounds and the remaining setup choices. We kept the released values wherever they existed, so that the run reflects the artifact as published rather than our preferences. We intend this as an audit aid rather than an accusation: underspecification is common, but here it coincides with the failure modes.

Table 3: Paper-vs-release underspecifications we resolved. "Released" marks a value shipped in the released configuration that we kept; "ours" marks a choice we had to make.

Design choice	In paper?	Value used	Note
Scaling-AE decoder hidden dim	no	1 (released)	load-bearing
PEM rollout policy	no	argmax (released)	load-bearing
Magnitude normalization at sampling	implied	commented out (released)	load-bearing
PGM / PEM training epochs	no	30 / 60 (ours; released 400 / 1000)	confound
PCDM diffusion steps	no	30 (ours; released 100)	confound
SISC k-means seed	no	42 (released)	shared seed
GOOG / ZC=F downstream split	S&P only	62.5% train (ours)	cross-asset parity
Initial segment for generation	no	first test segment / random (ours)	protocol switch

4.5 Confounds and what we cannot rule out

We stress that the evidence above concerns the released artifacts as configured and trained in our hands rather than the method in principle. Two confounds prevent a stronger claim. First, no trained checkpoints are shipped with the release, so every reproduction retrains from the released scripts and we cannot compare against the authors' own generations. Second, our training budget is reduced relative to the released defaults (PGM 30 vs. 400 epochs, PEM 60 vs. 1000, PCDM 30 vs. 100 steps), a compute-driven choice that a reviewer can reasonably challenge. Three non-exclusive explanations remain that we have not fully separated: undertraining, an unlucky seed, and the one-unit decoder capacity. A full-budget retrain across seeds would falsify or confirm the undertraining hypothesis; we flag it as the natural next experiment and do not assert a method-level conclusion without it. Given how unusual a one-unit decoder hidden state is for a published generative model, it would also be reasonable to ask the authors directly whether that value, and the commented-out magnitude normalization, correspond to the configuration behind the paper's headline results.

5 Replication results

5.1 TATR depends on the synthetic-series protocol

Huang et al. report that appending 100 synthetic years reduces one-day-ahead MAPE by 17.9%, 15.3%, and 17.4% on S&P 500, GOOG, and ZC=F, respectively. Our current S&P 500 evidence has three layers. First, the stored result matrices do not support that trend for the documented `authors` independent-block protocol: the stored S&P 500 `authors` curve finishes 208.4% worse than its baseline, while the continuous-trajectory curve finishes 71.6% better. Second, a one-seed protocol search found that continuous-rollout variants finish 76.5–85.4% better, whereas independent fixed blocks finish 96.5% worse. Third, Table 4 reports the primary 30-seed batch across the three protocols.

In that batch, the continuous chunked rollout reproduces a strong paper-like reduction in downstream MAPE, while both independent-block protocols worsen it. A single lucky seed does not drive this: across all 30 seeds the final continuous-chunked change stays below baseline (range $[-89.1\%, -21.2\%]$), whereas independent fixed blocks range $[+17.0\%, +438.3\%]$ and random-init blocks $[+3.9\%, +263.3\%]$. Figure 3 shows the same contrast: continuous trajectories cross below the real-only baseline, while independent blocks worsen as augmentation increases.

Table 4: S&P 500 TATR summary over 30 seeds. Percentages are relative to each protocol's no-augmentation baseline; lower MAPE is better. "Best change" is the minimum over the augmentation sweep.

Protocol	Baseline	Best change	Final change
Independent fixed blocks (`authors`)	0.0611	$+0.0\%$	$+208.4\%$
Continuous chunked (`split`)	0.0620	-77.6%	-72.2%
Random-init blocks (`random_init`)	0.0611	$+0.0\%$	$+68.2\%$

Synthetic price levels explain the effect. Continuous trajectories start near 1253 and end near 7085 (mean level 4138); independent blocks start from the same level but repeatedly reset, ending near 1249 (mean 1214). Continuous rollouts thus drift the near-linear ramp through the scale of the upward-trending S&P 500 test period, so percentage error can fall simply because synthetic and real price levels overlap, not because the dynamics are informative. Independent blocks stay near the initial level, the overlap never happens, and error grows. Read with Section 4, this looks like a scale effect rather than useful augmentation. Cross-asset evidence below provides the cleaner test.

Train-on-Mixture-Test-on-Real (TMTR) controls, which train the forecaster on a fixed real-plus-synthetic mixture rather than sweeping the amount of augmentation, make the replication conclusion more conservative. In our TMTR runs, the reference curve reaches a best mean improvement of 26.8% but finishes 359.7% worse; a continuous offset-30 control reaches a best improvement of 50.0% and finishes 17.8% better; a continuous offset-50 control finishes 752.0% worse. So the evidence does not say "synthetic data always helps." A narrower reading fits: a particular continuous TATR price-level rollout can create large apparent gains, while other augmentation and evaluation choices do not support the same conclusion.

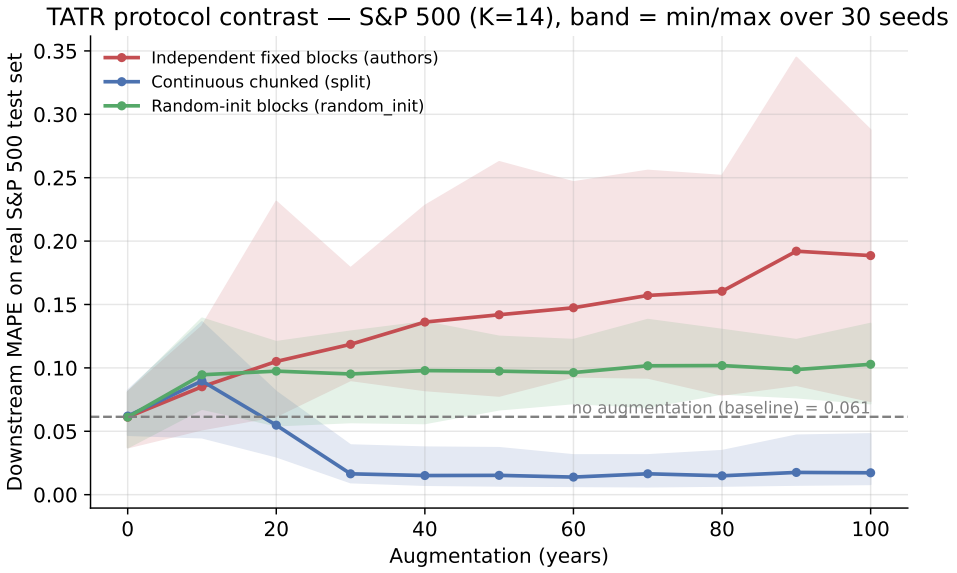

Figure 3: TATR outcome changes sharply with the synthetic-series protocol. Continuous trajectories can look paper-like, while independent blocks worsen with augmentation. A dashed line marks the original S&P 500 100-year TATR target, and shaded bands show min/max across seeds.

We make no claim of misconduct; the available generation details do not pin down the downstream result uniquely. Once the generator is seen to be degenerate, though, the simplest reading of the S&P 500 gain is a scale coincidence rather than augmentation that works under the right protocol. Either way, a reproducible benchmark should treat the Markov-chain rollout, initialization, block slicing, scaler fitting, and warm-up windows as first-class parameters, and should not rest a positive conclusion on downstream MAPE for a single trending asset.

GOOG and ZC=F test the coincidence reading most directly. We regenerated the stated real-asset SISC and architecture artifacts for both with $K = 11$, $l_{\min} = 10$, and $l_{\max} = 21$. A strict author-faithful downstream TATR rerun is blocked by the released S&P-style split logic, which consumes $252 \times 5 = 1260$ held-out initialization points whereas the GOOG and ZC=F 80/20 held-out windows contain 744 and 963 points, so we adapted the split while keeping the proportions comparable. This changes the absolute MAPE level but not the sign of the augmentation effect, since every percentage is computed within the same split relative to that protocol's own no-augmentation baseline. Under that within-split comparison the effect is negative on both assets and for both protocol families. Independent fixed blocks (`authors`) finish 1003% worse on GOOG and 75% worse on ZC=F. Nor does the continuous chunked (`split`) rollout that appears to help on S&P 500 transfer (Figure 4): its best point over the whole sweep equals the no-augmentation baseline (it never beats it), and it finishes 92% worse on GOOG and 179% worse on ZC=F. If the generator carried useful structure, the protocol would not flip sign across assets; a gain confined to the one strongly trending asset points to a scale coincidence, not augmentation value. We therefore treat GOOG and ZC=F as covered for SISC/architecture setup and real-asset diagnostics rather than author-faithful downstream replications, but the within-split sign does not depend on author-faithfulness.

5.2 SISC toy-data replication gap

We also stress-tested SISC on the Appendix B.3 simulated-data setting. Two metrics matter here. Per-unit DTW is the dynamic-time-warping distance between a recovered pattern centroid and its target shape, so lower is better. Jaccard is a segmentation-agreement measure, $|A \cap B|/|A \cup B|$, between the recovered set of segment boundaries A and the ground-truth set B: it equals 1.0 when the two segmentations coincide and 0 when they share no boundary, and we report a tolerance-2 variant that counts a recovered boundary as matched if it falls within two time steps of a true one. Huang et al. report one-pattern per-unit DTW/Jaccard of $0.009/0.938$ and multi-pattern average per-unit DTW/Jaccard of $0.01/0.784$. On our derived one-pattern runs, the best available DTW is 0.0202

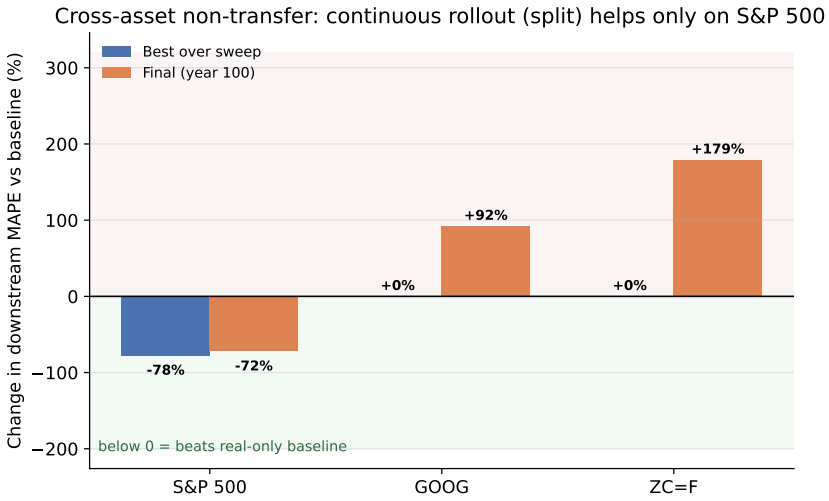

Figure 4: Cross-asset non-transfer of the continuous chunked (`split`) rollout. Bars show the change in downstream MAPE relative to each asset's no-augmentation baseline (below zero beats the real-only baseline). The rollout helps only on S&P 500 (best -78%, final -72%); on GOOG and ZC=F it never beats baseline and finishes $+92\%$ and $+179\%$. A protocol carrying genuine structure would not flip sign across assets.

and the best tolerance-2 boundary-Jaccard proxy is 0.8693. Across 18 successful multi-pattern runs varying seed, $l_{\max}$, and maximum iterations, our best average per-unit DTW was 0.0321, and our best author-style interval IoU (intersection over union of recovered and true segment intervals) was 0.6418. Figure 5 summarizes the multi-pattern gap.

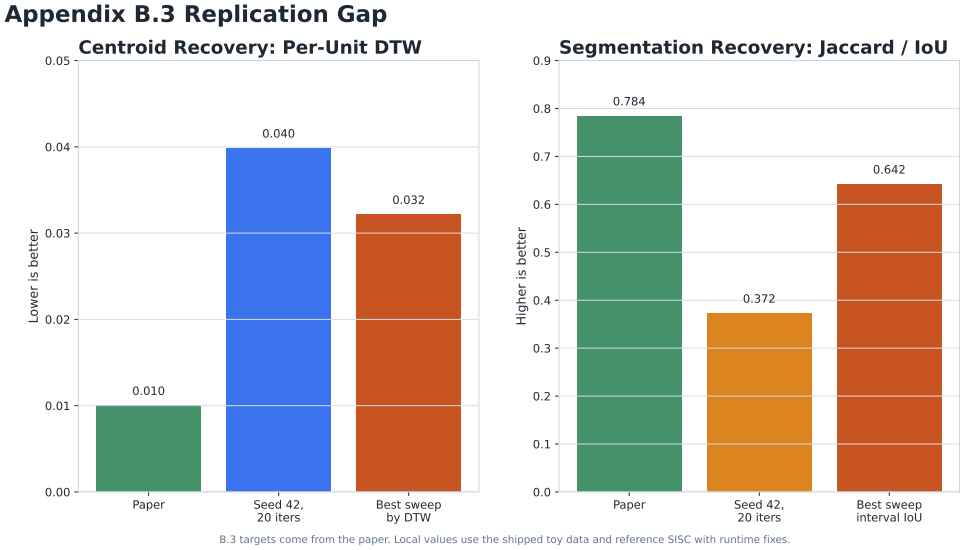

Figure 5: Appendix B.3 SISC replication gap on the toy data. Best replication runs do not reach the reported centroid or segmentation targets.

Increasing SISC iterations from 10 to 20 and changing $l_{\max}$ from 20 to 21 did not close the gap. Our best strict boundary Jaccard at tolerance 2 was 0.3608, and the best reference-code-style interval IoU (intersection over union of recovered and true segment intervals) was 0.6418. Our best centroid metric remained about $3.2\times$ the reported target. We could not reproduce the exact toy benchmark, though this says nothing about the general idea of SISC: the released tree does not expose the exact one-pattern construction, synthetic generator, standard pattern arrays, seed state, or metric

implementation used to produce Appendix B.3. More broadly, helper defaults matter: warm-up windows, augmentation scales, plotting conventions, and interval-overlap metrics can all change the apparent result if they go unaudited.

6 Predictable-drift extension

6.1 Diagnostic

Stylized facts are necessary but not sufficient for finance. A generator can reproduce heavy tails and volatility clustering while encoding a simple conditional mean, such as $r_{t+1} = \rho r_t + \epsilon_{t+1}$. Marginal tests may not detect this, but a trading signal using r_t can. We therefore treat the drift check as a fixed, predeclared diagnostic rather than a learned discriminator: the chronological split rule, feature map, train-only standardization, null construction, and companion stylized-fact summaries are fixed before evaluating synthetic outcomes.

Motivated by the martingale-difference intuition behind weak-form market efficiency [Fama, 1970], we fix a small information set

$$\omega_t = (1,\ r_t,\ r_{t-1},\ |r_t|,\ |r_{t-1}|)^\top \tag{3}$$

and a bounded linear class

$$g_b(\mathcal{F}_t) = b^\top \omega_t, \qquad \|b\|_2 \le 1. \tag{4}$$

The population predictable-drift violation is

$$\delta = \sup_{\|b\|_2 \le 1} \left| \mathbb{E}\left[r_{t+1} b^\top \omega_t \right] \right|. \tag{5}$$

Let $m = \mathbb{E}[r_{t+1}\omega_t]$. By Cauchy-Schwarz, Equation 5 has the closed form

$$\delta = \|m\|_2 . \tag{6}$$

The empirical estimator is therefore

$$\hat{\delta} = \left\| \frac{1}{T-2} \sum_{t=2}^{T-1} r_{t+1}\omega_t \right\|_2 . \tag{7}$$

This statistic decomposes into interpretable components: unconditional drift, first-lag return predictability, second-lag return predictability, and two volatility-conditioned mean terms. In our implementation, returns are standardized using training-split mean and standard deviation only; the constant feature remains equal to one. We compare real data, synthetic data, and a block-shuffled null that preserves the return multiset and some local dependence while disrupting longer chronological alignment.

6.2 Real-asset calibration

Before using $\hat{\delta}$ as a rejection statistic, we calibrate it on real held-out assets. For S&P 500, GOOG, and ZC=F, we start from raw close-price CSVs, use the first 80% of simple returns for standardization, and evaluate the last 20%. Our null distribution uses 500 replications of 21-day block shuffling on the held-out returns. Table 5 shows that all three real held-out paths lie inside their asset-specific 5–95% null bands, while retaining heavy tails and volatility clustering. So the diagnostic does not simply reject real financial data for being non-Gaussian.

6.3 Controlled predictive-alpha test

We next test whether larger $\hat{\delta}$ corresponds to exploitable predictability. In each of 500 seeds, we simulate a zero-mean heavy-tailed GARCH-like path, keep the evaluation-period magnitude path $|r_t|$ fixed, and replace only the signs. An iid-sign control has no intentional sign predictability, while Markov-sign samples have sign persistence p. We fit a ridge linear predictor on the first half of each evaluation path using the same feature map ω_t, then evaluate one-step prediction and a sign strategy on the second half.

Table 5: Real-asset predictable-drift calibration. Null band is the 5–95% range from 500 block-shuffled held-out paths. ACF($|r|$) is the lag-1 autocorrelation of absolute returns (a volatility-clustering measure) and Ex. kurt. the excess kurtosis of returns.

Asset	Test obs.	Real $\hat{\delta}$	Null 5–95%	ACF($\|r\|$)	Ex. kurt.
S&P 500	2018	0.0297	[0.0222, 0.0303]	0.2165	3.2110
GOOG	755	0.0575	[0.0403, 0.0629]	0.0935	6.1142
ZC=F	953	0.0173	[0.0150, 0.0259]	0.1250	2.0247

Table 6: Controlled predictive-alpha test over 500 seeds. Magnitude paths are fixed within seed; only sign dependence changes.

Sample	p	$\hat{\delta}$	OOS corr.	Dir. acc.	Sharpe (sd)
Real evaluation path	–	0.0976	0.0004	0.5025	0.07 (0.77)
IID signs	–	0.0941	0.0036	0.5020	0.09 (0.74)
Markov signs	0.65	0.2088	0.1482	0.5968	2.33 (0.96)
Markov signs	0.75	0.3435	0.2786	0.6952	4.80 (1.05)
Markov signs	0.85	0.5126	0.4112	0.7976	7.72 (1.18)
Markov signs	0.90	0.6164	0.4811	0.8483	9.43 (1.33)

Table 6 and Figure 6 show the result. Both the iid-sign and real controls have near-zero out-of-sample correlation, near-50% directional accuracy, and near-zero strategy Sharpe. At $p = 0.85$, the hidden-drift sample has mean $\hat{\delta} = 0.5126$, out-of-sample correlation 0.4112, directional accuracy 79.76%, and mean annualized Sharpe 7.72 with seed standard deviation 1.18. Across all controlled samples, pooled $\hat{\delta}$ and out-of-sample Sharpe have correlation 0.6921. So $\hat{\delta}$ tracks artificial alpha rather than serving as a passive reporting statistic.

6.4 Ordered FTS audit and incorporated selection

Applying this to FTS-Diffusion is direct: report Equation 7 on ordered samples from each rollout protocol, and do not select a protocol only because it improves TATR if its $\hat{\delta}$ lies outside the real/null band. We regenerated six S&P 500 synthetic paths per protocol from stored FTS-Diffusion checkpoints, using 100 ordered 252-day blocks per seed and excluding artificial transitions between independent blocks. Table 7 shows that continuous rollouts stay inside the S&P 500 null band in all six seeds, while independent fixed blocks violate it in all six. Under this statistic the audit does not reject the continuous rollout that produces paper-like TATR gains; it flags the independent reset protocol as both downstream-harmful and predictably drifted.

We next test whether the same principle helps choose augmentations in a controlled benchmark. For each of 1000 seeds, all candidate augmentations shared the same magnitude path but had different sign dynamics. A marginal stylized selector minimized a distance over volatility, tail, quantile, and absolute-return autocorrelation summaries; the drift-incorporated selector minimized

$$d_{\text{styl}} + 0.25 \left[\hat{\delta}/q_{.95}^{\text{real}} - 1\right]_+^2, \tag{8}$$

where $q_{.95}^{\text{real}}$ is the real block-shuffle null cutoff. Each selected augmentation trained the same one-step ridge predictor on a 252-day real initialization plus synthetic returns, with performance measured on a final held-out real path that the drift-penalized selector does not see.

Table 8 shows that incorporating the diagnostic changes the selected augmentation, not just the reporting. Relative to stylized-only selection, the drift-penalized selector reduces high-persistence choices from 39.3% to 0.2%, cuts real-null violations from 76.0% to 34.6%, and lowers held-out normalized MSE by 0.0365 on average, with a 60.8% paired win rate. If the statistic is incorporated during training rather than selection, use it as a validation-selected penalty

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{FTS}} + \lambda \hat{\delta}^2_{\text{synthetic}}. \tag{9}$$

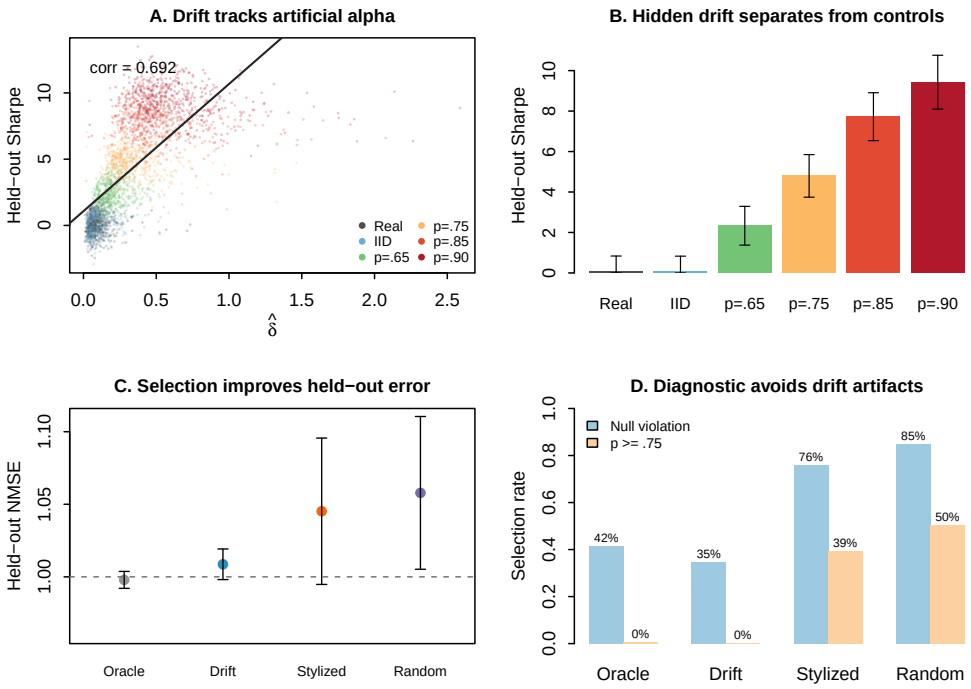

Figure 6: Drift versus no-drift evidence from the larger controlled experiments. The diagnostic tracks held-out artificial alpha, separates hidden-drift sign processes from real and iid controls, and improves augmentation selection by avoiding null violations and high-persistence candidates.

Table 7: Ordered S&P 500 FTS-Diffusion drift audit over six seeds and 25,200 generated prices per seed. The cutoff is the S&P 500 held-out block-shuffle null 95th percentile, $q_{.95} = 0.0303$.

Protocol	$\hat{\delta}$	$\hat{\delta}/q_{.95}$	Violations	Price std.
Continuous rollout	0.0263	0.87	0/6	1701.5
Independent fixed blocks	0.0370	1.22	6/6	24.5

Computing the penalty requires ordered return sequences. Applying it across randomly shuffled mini-batches would destroy the time ordering and should only be logged as a proxy, not as the final diagnostic.

7 Limitations and broader impact

Our replication is constrained by the information and compute available for this study. Most importantly, the degeneracy analysis of Section 4 concerns the released artifacts as configured and as trained in our hands: no checkpoints are shipped, and our training budget is reduced relative to the released defaults. We have not run the full-budget, multi-seed retrain that would separate undertraining from an architectural cause, so we deliberately phrase those findings as observations about the release rather than conclusions about the method. Some runs use reference checkpoints rather than a full fresh retraining of all modules, and the ordered FTS-Diffusion drift audit is limited to the S&P 500 checkpoints and the fixed diagnostic in Equation 7. Train-time validation selection of λ remains to be run before claiming an optimized drift-regularized generator. GOOG and ZC=F have shorter histories than S&P 500, so the downstream split used for S&P 500 is not directly portable without changing the question being asked; we mitigate this by comparing within split, relative to each protocol's own baseline.

Table 8: Drift-incorporated augmentation selection over 1000 seeds. Lower normalized mean squared error (NMSE) is better. High-p is the fraction of selected Markov-sign candidates with persistence $p \geq 0.75$. Oracle uses held-out test MSE and is only a reference upper bound.

Selector	Null viol.	High-p	Styl. score	Test NMSE	Excess
Oracle test-MSE	41.5%	0.5%	0.2135	0.9979	-0.0319
Drift-penalized	34.6%	0.2%	0.0520	1.0087	-0.0211
Stylized-only	76.0%	39.3%	0.0344	1.0452	$+0.0154$
Random	84.8%	50.2%	0.3106	1.0579	$+0.0281$

Our predictable-drift diagnostic covers a small hypothesis class by design. It tests only the five-dimensional linear feature map ω_t, so it can miss nonlinear or longer-memory predictability and does not constitute a proof of no-arbitrage. Its compensating strength is reproducibility, since the statistic is simple, closed-form, and hard to tune after seeing the result.

Synthetic financial data can be beneficial for stress testing, augmentation, and reproducibility, but it can also create misleading evidence if artificial predictability is mistaken for real market structure. Any release of synthetic paths should clearly label them as generated data, document the training period and generation protocol, and avoid implying that downstream forecasting gains translate into tradable performance.

8 Conclusion

FTS-Diffusion has an appealing inductive bias for financial time series: variable-length motifs, pattern-conditioned diffusion, and learned motif evolution. Our reproduction attempt, however, did not yield a working generator from the released artifacts. In our hands the released sampling pipeline collapses to a near-linear ramp built from one repeated motif, the authors' own return-distribution test cleanly flags that output as far from real, and the one downstream setting that appears to improve forecasting does so only on a single strongly trending asset and fails to transfer to GOOG or ZC=F. We read that pattern as a scale coincidence rather than useful augmentation. We do not overstate it: without the authors' checkpoints and full training budget we cannot yet separate the degeneracy from undertraining, so we report empirical observations about the release, not a verdict on the method. One methodological point holds regardless: realistic-looking paths and a favorable downstream MAPE on a trending series are weak evidence on their own. Future synthetic-finance benchmarks should report exact rollout protocols, direct generator diagnostics such as the return-distribution and predictable-drift checks used here, and cross-asset transfer, rather than resting positive conclusions on a single asset.

References

Rama Cont. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.

Eugene F. Fama. Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2):383–417, 1970.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

Hongbin Huang, Minghua Chen, and Xiao Qiao. Generative learning for financial time series with irregular and scale-invariant patterns. In *International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=CdjnzWsQax`.

Francois Petitjean, Alain Ketterlin, and Pierre Gancarski. A global averaging method for dynamic time warping, with applications to clustering. *Pattern Recognition*, 44(3):678–693, 2011.

Hiroaki Sakoe and Seibi Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49, 1978.

Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based diffusion models for probabilistic time series imputation. In *Advances in Neural Information Processing Systems*, volume 34, pages 24804–24816, 2021.

Magnus Wiese, Robert Knobloch, Ralf Korn, and Peter Kretschmer. Quant GANs: Deep generation of financial time series. *Quantitative Finance*, 20(9):1419–1440, 2020.

Tianlin Xu, Li K. Wenliang, Michael Munn, and Beatrice Acciaio. COT-GAN: Generating sequential data via causal optimal transport. In *Advances in Neural Information Processing Systems*, volume 33, pages 8798–8809, 2020.

Jinsung Yoon, Daniel Jarrett, and Mihaela van der Schaar. Time-series generative adversarial networks. In *Advances in Neural Information Processing Systems*, volume 32, 2019.